

Experiment No. 7

Motion Estimation using Optical Flow Algorithms in Video Sequences

Aim: To implement optical flow algorithms for estimating the motion of objects between consecutive video frames and analyze motion patterns using Python and OpenCV.

Algorithms: Lucas-Kanade Sparse Optical Flow and Farneback Dense Optical Flow

Libraries: OpenCV, NumPy, Matplotlib

This notebook is designed for **Google Colab**. It uses a short public sample video automatically, so no manual video upload is required. An optional upload cell is also provided.

```
# Install/import required libraries
!pip -q install opencv-python-headless

import cv2
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import HTML, display
import urllib.request
import time

print("OpenCV:", cv2.__version__)
print("NumPy:", np.__version__)

OpenCV: 5.0.0
NumPy: 2.1.3
```

1. Load a Video Sequence

```
# Download a short sample video
video_url =
"https://raw.githubusercontent.com/opencv/opencv/master/samples/data/
vtest.avi"
video_path = "/content/vtest.avi"

urllib.request.urlretrieve(video_url, video_path)

cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
```

```

        raise ValueError("Video could not be opened.")

fps = cap.get(cv2.CAP_PROP_FPS)
frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

print(f"Resolution: {width} x {height}")
print(f"FPS: {fps:.2f}")
print(f"Total frames: {frame_count}")
print(f"Duration: {frame_count/fps:.2f} seconds")
cap.release()

Resolution: 768 x 576
FPS: 10.00
Total frames: 795
Duration: 79.50 seconds

```

Optional: Upload Your Own Video

```

# Run this cell only if you want to use your own video.
# The uploaded file will replace video_path.

from google.colab import files

# uploaded = files.upload()
# video_path = "/content/" + next(iter(uploaded))
# print("Using:", video_path)

```

2. Read Consecutive Frames and Convert to Grayscale

```

cap = cv2.VideoCapture(video_path)

ret, frame1 = cap.read()
ret2, frame2 = cap.read()

if not ret or not ret2:
    cap.release()
    raise ValueError("Could not read two consecutive frames.")

gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(12,4))

plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB))
plt.title("Frame 1")

```

```
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(frame2, cv2.COLOR_BGR2RGB))
plt.title("Frame 2")
plt.axis("off")

plt.tight_layout()
plt.show()

cap.release()
```



3. Lucas-Kanade Sparse Optical Flow

```
# Shi-Tomasi corner detection selects good feature points.
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

p0 = cv2.goodFeaturesToTrack(gray1, mask=None, **feature_params)

if p0 is None:
    raise ValueError("No suitable feature points were found.")

lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10,
0.03)
)

start = time.perf_counter()
```

```

p1, st, err = cv2.calcOpticalFlowPyrLK(gray1, gray2, p0, None,
**lk_params)
lk_time = time.perf_counter() - start

good_new = p1[st == 1]
good_old = p0[st == 1]

print("Initial feature points:", len(p0))
print("Successfully tracked points:", len(good_new))
print(f"Lucas-Kanade execution time: {lk_time*1000:.3f} ms")

Initial feature points: 56
Successfully tracked points: 56
Lucas-Kanade execution time: 4.250 ms

```

4. Visualize Lucas-Kanade Motion Vectors

```

lk_visual = cv2.cvtColor(frame2, cv2.COLOR_BGR2RGB).copy()

for new, old in zip(good_new, good_old):
    x1, y1 = new.ravel()
    x0, y0 = old.ravel()

    # Draw motion line and endpoint
    cv2.line(lk_visual, (int(x0), int(y0)), (int(x1), int(y1)), (255,
0, 0), 2)
    cv2.circle(lk_visual, (int(x1), int(y1)), 3, (0, 255, 0), -1)

plt.figure(figsize=(10,6))
plt.imshow(lk_visual)
plt.title("Lucas-Kanade Sparse Optical Flow")
plt.axis("off")
plt.show()

```

Lucas-Kanade Sparse Optical Flow



5. Track Feature Trajectories Across Multiple Frames

```
cap = cv2.VideoCapture(video_path)

ret, old_frame = cap.read()
if not ret:
    cap.release()
    raise ValueError("Could not read the first frame.")

old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)

trajectory = np.zeros_like(old_frame)
frames_to_process = 40
processed = 0

while processed < frames_to_process:
    ret, frame = cap.read()
    if not ret:
        break
```

```

frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

if p0 is None or len(p0) == 0:
    break

p1, st, err = cv2.calcOpticalFlowPyrLK(
    old_gray, frame_gray, p0, None, **lk_params
)

if p1 is None:
    break

good_new = p1[st == 1]
good_old = p0[st == 1]

for new, old in zip(good_new, good_old):
    a, b = new.ravel()
    c, d = old.ravel()
    cv2.line(trajecory, (int(a), int(b)), (int(c), int(d)), (255,
0, 0), 2)

old_gray = frame_gray.copy()
p0 = good_new.reshape(-1, 1, 2)
processed += 1

cap.release()

trajecory_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
trajecory_rgb = cv2.add(trajecory_rgb, cv2.cvtColor(trajecory,
cv2.COLOR_BGR2RGB))

plt.figure(figsize=(10,6))
plt.imshow(trajecory_rgb)
plt.title("Lucas-Kanade Feature Trajectories")
plt.axis("off")
plt.show()

print("Frames processed for trajectory:", processed)

```

Lucas-Kanade Feature Trajectories



Frames processed for trajectory: 40

6. Farneback Dense Optical Flow

```
start = time.perf_counter()

flow = cv2.calcOpticalFlowFarneback(
    gray1, gray2, None,
    pyr_scale=0.5,
    levels=3,
    winsize=15,
    iterations=3,
    poly_n=5,
    poly_sigma=1.2,
    flags=0
)

farneback_time = time.perf_counter() - start
```

```
magnitude, angle = cv2.cartToPolar(flow[..., 0], flow[..., 1])

print(f"Farneback execution time: {farneback_time*1000:.3f} ms")
print(f"Average motion magnitude: {np.mean(magnitude):.4f}")
print(f"Maximum motion magnitude: {np.max(magnitude):.4f}")
```

```
Farneback execution time: 195.650 ms
Average motion magnitude: 0.2734
Maximum motion magnitude: 11.5824
```

7. Color-Coded Dense Motion Visualization

```
# HSV representation:
# Hue -> direction
# Saturation -> strength
# Value -> motion magnitude

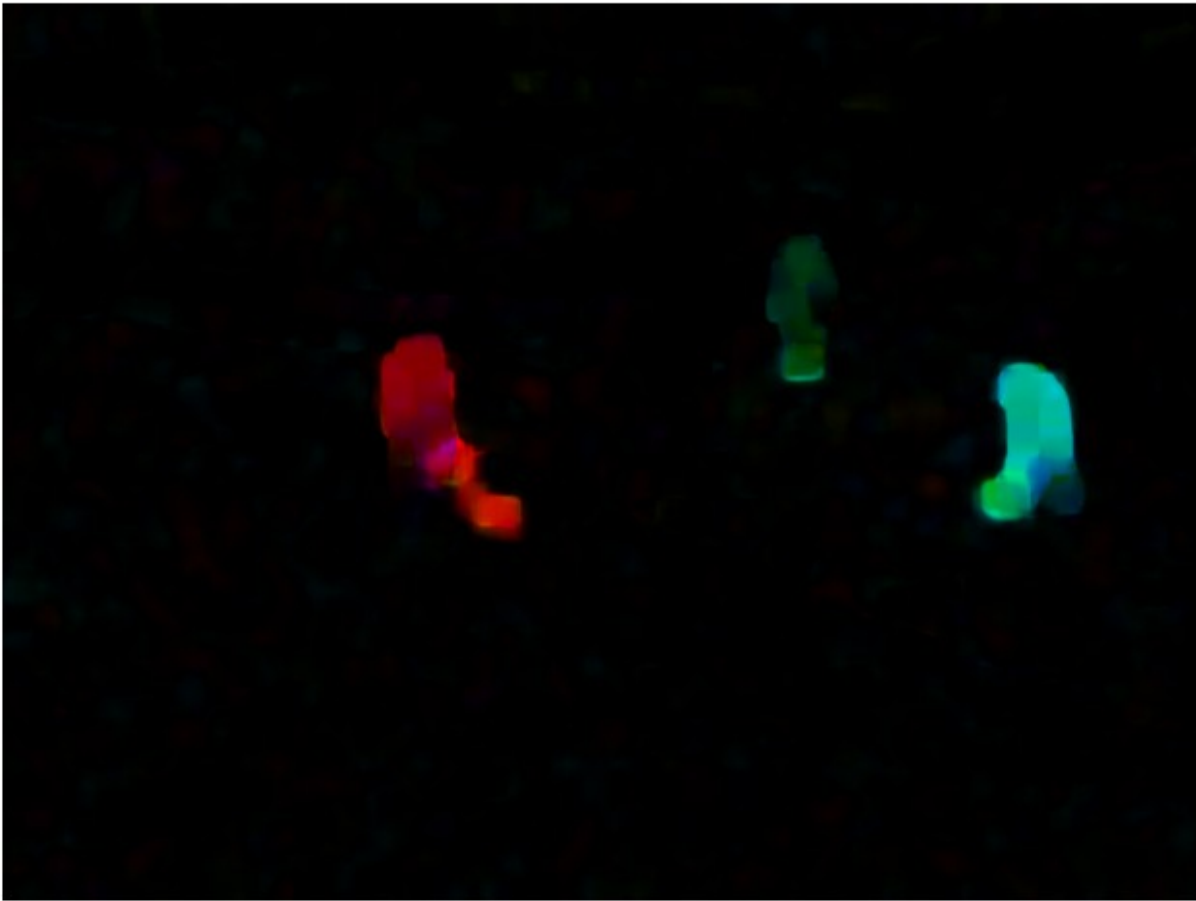
hsv = np.zeros_like(frame1)
hsv[..., 1] = 255

hsv[..., 0] = angle * 180 / np.pi / 2
hsv[..., 2] = cv2.normalize(magnitude, None, 0, 255, cv2.NORM_MINMAX)

dense_bgr = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
dense_rgb = cv2.cvtColor(dense_bgr, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(10,6))
plt.imshow(dense_rgb)
plt.title("Farneback Dense Optical Flow")
plt.axis("off")
plt.show()
```

Farneback Dense Optical Flow



8. Compare Sparse and Dense Optical Flow

```
plt.figure(figsize=(16,6))

plt.subplot(1,2,1)
plt.imshow(lk_visual)
plt.title("Lucas-Kanade: Sparse Motion")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(dense_rgb)
plt.title("Farneback: Dense Motion")
plt.axis("off")

plt.tight_layout()
plt.show()
```

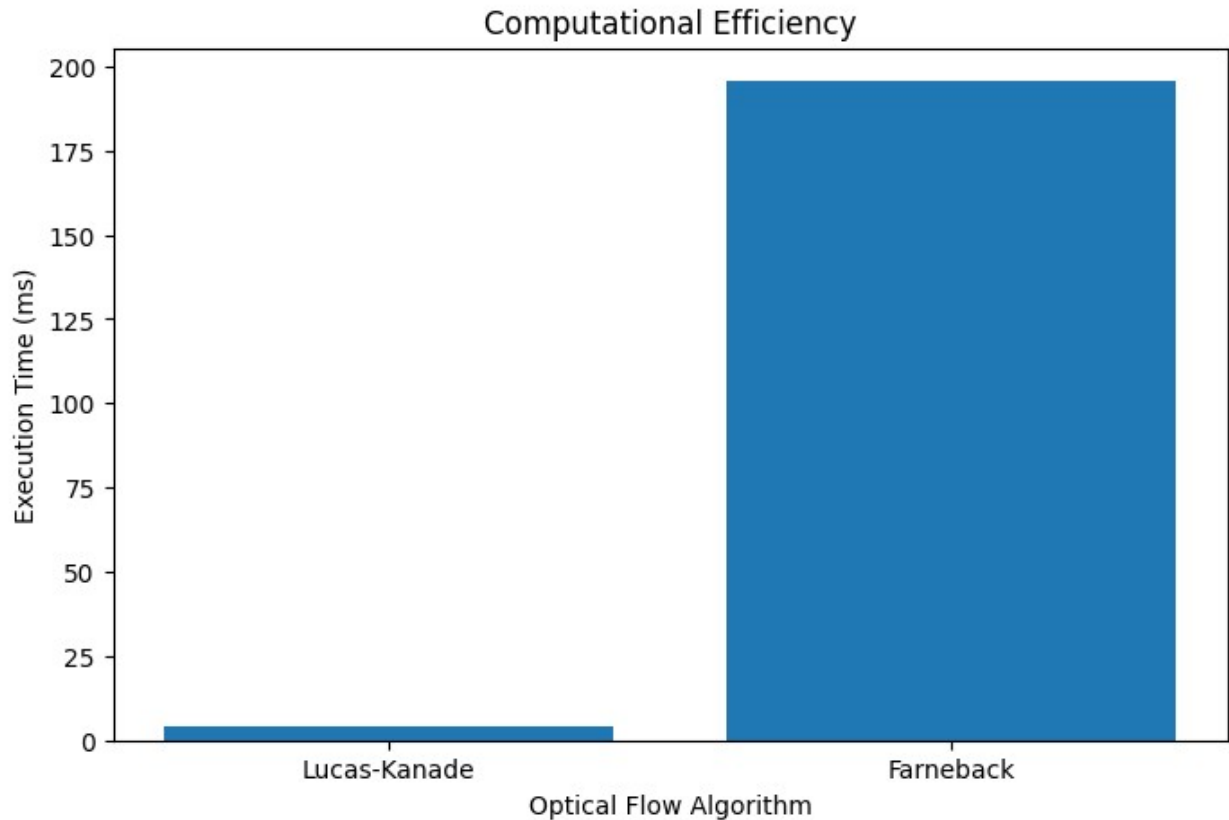


9. Computational Efficiency Comparison

```
methods = ["Lucas-Kanade", "Farneback"]
times_ms = [lk_time * 1000, farneback_time * 1000]

plt.figure(figsize=(8,5))
plt.bar(methods, times_ms)
plt.ylabel("Execution Time (ms)")
plt.xlabel("Optical Flow Algorithm")
plt.title("Computational Efficiency")
plt.show()

print(f"Lucas-Kanade: {times_ms[0]:.3f} ms")
print(f"Farneback: {times_ms[1]:.3f} ms")
```



Lucas-Kanade: 4.250 ms
Farneback: 195.650 ms

10. Observations and Analysis

Lucas-Kanade Sparse Optical Flow

- Tracks selected feature points rather than every pixel.
- Computationally efficient because only a limited number of points are processed.
- Useful for object tracking, feature tracking, camera motion estimation, and real-time applications.
- Performance depends strongly on the quality and number of detectable feature points.

Farneback Dense Optical Flow

- Estimates motion for nearly every pixel in the image.
- Produces a complete motion field and is useful for understanding motion across an entire scene.
- Usually requires more computation than sparse Lucas-Kanade.
- Color-coded visualization makes direction and magnitude of motion easy to interpret.

Effect of Scene Conditions

- **Fast object motion:** Can produce larger displacements and tracking errors if the algorithm's search/pyramid parameters are insufficient.
- **Lighting changes:** Can violate the brightness-constancy assumption and reduce accuracy.
- **Camera motion:** Causes widespread optical flow, including background motion.
- **Textureless regions:** Provide few reliable features for sparse tracking.
- **Occlusion:** Makes correspondence between frames difficult.

Questions and Answers

1. What is Optical Flow? How is it used in computer vision?

Optical flow is the apparent motion of image pixels or visual features between consecutive video frames. It is used to estimate object motion, camera movement, scene dynamics, tracking, navigation, and activity.

2. Explain the working principle of Lucas-Kanade Optical Flow.

Lucas-Kanade assumes that pixels within a small neighborhood have approximately the same motion. It uses image gradients and the brightness-constancy constraint to form a set of linear equations and solves them in a least-squares sense for the motion vector of selected feature points. Pyramidal Lucas-Kanade can handle larger movements by estimating motion at multiple scales.

3. What is Dense Optical Flow? How does it differ from Sparse Optical Flow?

Dense optical flow estimates a motion vector for almost every pixel in the image. Sparse optical flow estimates motion only at selected points, usually strong features. Sparse flow is generally faster and lighter, while dense flow provides more complete scene motion information.

4. Compare Lucas-Kanade and Farneback optical flow.

Lucas-Kanade: sparse, feature-based, relatively fast, suitable for tracking selected points.

Farneback: dense, estimates motion over the image, more computationally expensive, suitable for analyzing complete motion fields.

5. What assumptions are made while computing optical flow?

Common assumptions are:

1. **Brightness constancy:** the appearance/intensity of a moving point remains approximately constant between frames.
2. **Small motion:** displacement between consecutive frames is small enough for local approximations.

3. **Spatial smoothness/local consistency:** nearby pixels generally have similar motion, especially in local regions.

6. What factors can affect the accuracy of optical flow?

Object speed, frame rate, image noise, illumination changes, shadows, reflections, occlusion, motion blur, textureless regions, camera motion, large displacement, and incorrect algorithm parameters can affect accuracy.

7. Mention five real-world applications of optical flow.

1. Autonomous driving
2. Video surveillance
3. Robot navigation
4. Sports analytics
5. Gesture/action recognition

8. Why are grayscale images generally used for optical flow?

Grayscale simplifies computation because optical-flow equations primarily use intensity changes and image gradients. It reduces the data dimensionality while retaining useful structural information such as edges and brightness variations.

9. What are the limitations of optical flow algorithms in real-world environments?

They can struggle with illumination changes, occlusions, motion blur, large displacements, repetitive patterns, textureless areas, transparency, reflections, and independently moving objects. Camera motion can also create flow across much of the image.

10. How does optical flow contribute to autonomous driving, video surveillance, and action recognition?

In **autonomous driving**, optical flow helps estimate motion of vehicles, pedestrians, obstacles, and camera/ego motion. In **video surveillance**, it helps detect and track moving objects and unusual activity. In **action recognition**, temporal motion patterns provide information about gestures and human activities that cannot be obtained from a single image alone.

Conclusion

The experiment successfully implemented Lucas-Kanade Sparse Optical Flow and Farneback Dense Optical Flow using Python and OpenCV. Lucas-Kanade efficiently tracks selected feature points and is suitable for real-time tracking tasks, whereas Farneback provides a dense motion field for more comprehensive scene analysis. Optical flow is therefore an important technique for dynamic scene understanding and motion-based computer vision applications.